

用第一性原理拆解 Agentic Coding：从理论到实操（下）

本文作者：小夏，TRAE 技术专家



在 Agentic Coding 实践中，提升效率的关键在于优化与 AI 的协作方式，而非寄望于无限的上下文窗口。核心策略是采用“短对话、精简上下文”的模式，将复杂任务拆解为专注的子对话，并借助“复利工程”将 bug 修复、代码审查等日常经验沉淀为可复用的项目知识库，使系统获得记忆并实现效率的持续增长。此外，改善开发者体验（如清晰文档、快速测试）具有双重价值，既能帮助人类开发者，也能显著提升 AI 的表现。最终，与 AI 的协作应被视为一门需要刻意练习的技能，通过不断实践，开发者可以成为驾驭 AI 的“专家型通才”，在更广阔的领域创造价值。

上一篇带大家了解了 LLM 的特性、Coding Agent 的实现原理以及常见问题后，我们总结出了一些最佳实践，帮助大家更高效地使用这些工具。

1. 短对话优于长对话

这可能是最重要的一条实践：**保持对话简短、专注，每个对话只做一件事。**

很多人认为更大的上下文窗口意味着更强的能力，可以把更多任务塞进一个对话里。但实际情况恰恰相反，最好的对话是短对话，它们只做一件事，并且只包含完成这件事所需的上下文。

为什么短对话更好？

当你往上下文里塞太多内容时，Agent 的表现就像喝醉了一样：它会开始犯错、跌跌撞撞、甚至开始和你争论，如果你继续喂它更多 token，它甚至会「吐」得你一身（产生大量无意义的输出）或者进入死循环。

对话越长，上下文窗口里就会积累越多与当前任务不太相关的内容。为了让 Agent 发挥最佳工况，你需要给它完成当前工作所需的上下文。

长对话不仅效果差，还更贵。每次发送消息时，整个上下文都会被发送给模型提供商。这意味着对话越长，新消息的成本就会指数级增长。而且长对话更容易因为消息间隔时间长而错过缓存窗口，导致费用飙升。所以，长对话既效果差，又花费高，尤其在以 tokens 消耗计费的套餐下面。

拆分对话，本质上是拆分任务，大任务应该被分解为小任务。这在 Agent 时代之前就是软件工程的最佳实践，现在看也依然如此，短对话让任务拆分这件事变得自然甚至有趣。就像小任务更容易管理一

样，小对话也更容易追踪：每个对话都有明确的目标，你可以轻松掌握整体进度。

2. 以对话为单位组织工作

如果把对话视为任务的基本单位，那么一个功能或 bug 修复就自然变成了一组相互关联的对话。

一个典型的工作流程

假设你要实现一个新功能，可以这样组织对话：

代码块

```
1  功能：用户登录后的会话管理
2
3  [对话 1] 调研现有代码结构
4      |—— 了解 auth 模块的实现
5      |—— 查看 session 管理的现状
6      |—— 输出：关键文件列表和当前架构理解
7
8  [对话 2] 实现基础功能
9      |—— 参考对话 1 的发现
10     |—— 实现核心的 session 保存逻辑
11     |—— 输出：基础实现代码
12
13 [对话 3] 添加错误处理
14     |—— 参考对话 2 的实现
15     |—— 增加边界情况处理
16
17 [对话 4] 编写测试
18     |—— 参考对话 2、3 的实现
19     |—— 添加单元测试和集成测试
20
21 [对话 5] 代码审查
22     |—— 检查实现是否符合项目规范
23     |—— 确认没有引入安全问题
24
25 [对话 6] 清理和重构
26     |—— 根据审查结果进行调整
```

每个对话都很短，只专注于一件事。它们加在一起，完成了整个功能的开发。

那对话之间如何共享上下文？

当你开始一个新对话时，可以通过以下方式传递必要的上下文：

- **引用之前对话的结论：**在新对话开头简要说明之前的发现或决策
- **利用 Git 状态：**让 Agent 查看 `git diff` 或检查最近的提交
- **使用项目文档：**将重要决策记录在 `AGENTS.md` 或类似文件中，Agent 每次都能读取

- **直接提及相关文件：**在新对话中 `#mention` 需要的文件

关键是：**不要试图在一个对话里完成所有事情。**每当你发现当前任务已经完成，或者对话开始变得混乱，就应该开始一个新对话。

3. 编写有效的项目配置文件

大多数 Coding Agent 都支持在项目根目录放置配置文件（比如 Rules 或者 Agent.md），这个文件会**自动注入到每一个对话中**，这意味着它是你影响 Agent 行为的杠杆支点，但这把双刃剑也很容易用错。

理解 Agent 的无状态本质

LLM 是无状态函数。它的权重在推理时是冻结的，不会随着使用而学习，Agent 对你代码库的全部了解，完全来自于你放进上下文窗口的 token，这有三个重要含义：

1. 每次新对话开始时，Agent 对你的代码库一无所知
2. 任何重要的项目信息都需要在每次会话中告诉它
3. 项目配置文件是实现这一点的首选方式

因此，你应该把这个文件视为**每次会话的入职培训文档**。

配置文件应该包含什么

一个好的项目配置文件应该回答三个问题：

- **WHAT（是什么）：**技术栈、项目结构、各模块的职责。这在 monorepo 中尤其重要，应该告诉 Agent 有哪些应用、哪些共享模块、每个部分是做什么的
- **WHY（为什么）：**项目的目的、设计决策的背景。为什么选择这个架构？为什么有些代码看起来不合理（比如历史债务）？
- **HOW（怎么做）：**如何运行项目、如何测试、如何验证改动。用 `bun` 还是 `npm`？测试命令是什么？

少即是多

这是最容易犯的错误：试图把所有可能需要的信息都塞进配置文件。

研究表明，前沿的思考模型大约能可靠地遵循 150-200 条指令，而 Coding Agent 的系统提示本身可能已经包含了约 50 条指令。这意味着你的配置文件应该**尽可能精简**，理想情况下只包含那些对所有任务都普遍适用的内容。

代码块

```
1  # 不好的做法：塞满各种可能用到的信息
2  ## 数据库 Schema 设计规范
3  (500 行详细规范...)
4
5  ## API 设计指南
6  (300 行规范...)
```

```

7
8  ## 代码风格指南
9  (200 行规范...)
10
11 # 好的做法：简洁 + 指向详细文档
12 ## 项目概述
13 这是一个 Next.js 电商平台，使用 PostgreSQL + Prisma。
14
15 ## 关键目录
16 - `apps/web`：前端应用
17 - `apps/api`：后端服务
18 - `packages/shared`：共享类型和工具
19
20 ## 开发命令
21 - `bun dev`：启动开发服务器
22 - `bun test`：运行测试
23 - `bun typecheck`：类型检查
24
25 ## 详细文档
26 根据任务需要，查阅以下文档：
27 - 数据库设计：`docs/database-schema.md`
28 - API 规范：`docs/api-guidelines.md`
29 - 代码风格：`docs/code-style.md`

```

一个经验法则：配置文件应该控制在 **300 行以内**，越短越好。有些团队的配置文件甚至不到 60 行。

渐进式披露

与其在配置文件中塞满所有信息，不如使用**渐进式披露**策略：

代码块

```

1  agent_docs/
2    |— building_the_project.md
3    |— running_tests.md
4    |— code_conventions.md
5    |— service_architecture.md
6    |— database_schema.md

```

在配置文件中列出这些文档并简要描述，让 Agent 根据当前任务决定读取哪些。这样，只有相关的信息才会进入上下文，避免不必要的干扰。

偏好指针而非副本：不要在文档中复制代码片段——它们很快会过时。使用 `file:line` 引用指向代码的权威位置。

不要让 Agent 做 Linter 的工作

很多人喜欢在配置文件中写详细的代码风格指南，这不是一个好的实践。

LLM 做格式检查既慢又昂贵，更重要的是，这些指南会增加指令数量，降低 Agent 对其他指令的遵循能力。

更好的做法：

- 使用真正的 linter 和 formatter（如 ESLint、Prettier、Biome）
- 配置自动修复，让工具处理格式问题
- 如果 Agent 产生了格式错误，让 linter 在后处理阶段修复它

LLM 是上下文学习者。如果你的代码库遵循一致的风格，Agent 通常会自动模仿这种风格，不需要你明确告诉它。

这是最高杠杆点，要认真对待

一行糟糕的代码就是一行糟糕的代码，一个糟糕的技术方案可能产生很多行糟糕的代码，而配置文件中的一行糟糕的指令会影响**每一个会话、每一个任务、每一个产出**。

代码块

1

杠杆效应：

2

3

配置文件 → 影响每个会话的行为

4

↓

5

研究/规划阶段 → 影响实现计划的质量

6

↓

7

实现阶段 → 影响最终代码的质量

花时间仔细考虑配置文件的每一行，这是你能做的 ROI 最高的投资之一。

4. 200K Token 足够了

当大家都在追求更大的上下文窗口时，一个反直觉的事实是：200K token 对于大多数任务来说已经绰绰有余了。

关键不在于你有多大的上下文窗口，而在于你如何使用它。一个 200K 的窗口，如果你用短对话的方式工作，可以支持你完成非常复杂的功能。因为虽然每个对话只有几十 K 到上百 K 个 token，但你可以开启 10 个、20 个甚至更多对话，它们加起来的总量远超任何单一上下文窗口。而且，由于每个对话都是从相对干净的状态开始，Agent 的表现会一直保持在最佳水平，而不是随着上下文膨胀而逐渐退化。

实践建议

- 当对话超过 80K-100K token 时，考虑开始新对话
- 完成一个独立的子任务后，开始新对话处理下一个任务
- 如果 Agent 开始表现出「醉酒」症状（重复、遗忘、偏离目标），立即开始新对话

- 把「开始新对话」视为正常工作流程的一部分，而不是「失败后的重试」

5. Compounding Engineering: 让系统自我改进

传统的 AI 编程是关于短期收益的：你给 prompt，它写代码，然后发布，然后从头开始。Every.to 提出的 **Compounding Engineering（复利工程）** 则是关于构建具有记忆的系统：每个 PR 都在教育系统，每个 bug 都成为永久的教训，每次代码审查都在更新 Agent 的默认行为。普通 AI 工程让你今天更高效，Compounding Engineering 让你之后的每一天都更高效。

核心理念：你不只是在解决问题，而是在教育系统

当你使用 Coding Agent 时，问自己一个问题：我是在解决今天的问题，还是在教系统？

- 每次修复 bug 时，如果不能防止同类问题再次发生，就只完成了一半
- 每次代码审查如果不能提取出可复用的教训，就是浪费时间
- 每次成功的工作流程如果不能被记录和复用，就会随着会话结束而消失

如何实践 Compounding Engineering

1. 将经验沉淀到项目文档

大多数 Coding Agent 都支持读取项目根目录下的特定文件，这是你指导系统的主要途径：

代码块

```
1  # AGENTS.md
2
3  ## 代码风格
4  - 使用 async/await 而非 Promise.then()
5  - 错误处理必须包含具体的错误类型
6  - 变量命名遵循 PR #234 确立的模式
7
8  ## 已知陷阱
9  - session 模块的 save() 方法是异步的，必须 await
10 - 不要在循环中调用 API，使用批量接口
11
12 ## 成功模式
13 - 新增 API 端点时，参考 PR #241 的错误处理方式
14 - 测试覆盖率要求参考 PR #219 的反馈
```

每次你发现一个重复出现的问题或一个有效的解决方案，就把它加入这个文件。Agent 在每次对话开始时都会读取它，自动应用这些经验。

2. 让 bug 修复产生长期价值

当你修复一个 bug 时，不要只是改代码。问自己：

- 这类问题能否通过添加 lint 规则来预防？

- 是否应该在 Rules 或者 AGENTS.md 中记录这个陷阱？
- 能否编写一个测试来防止回归？
- 代码审查清单是否需要更新？

一个真正的 bug 修复应该让同类问题再也不会发生。

3. 从代码审查中提取模式

每次你在审查中指出问题或提出建议，可以考虑：

- 这个反馈是否适用于未来的类似代码？
- 是否应该成为项目的编码规范？
- Agent 能否在下次自动应用这个改进？

如果答案是肯定的，就把它记录下来。让你的审查意见成为系统的永久知识，而不是一次性的对话。

4. 建立可复用的工作流程

当你找到一个有效的工作模式时，把它进行沉淀：

代码块

```
1  ## 工作流程：添加新的 API endpoint
2
3  1. 先编写接口测试（参考 tests/api/example.test.ts）
4  2. 实现端点，遵循 src/api/users.ts 的模式
5  3. 添加错误处理，使用 AppError 类
6  4. 更新 API 文档
7  5. 运行完整测试套件验证
```

下次你或 Agent 需要做类似的任务时，可以直接说「按照添加新 API endpoint 的工作流程来做」，系统已经知道该怎么做了。

复利效应

Compounding Engineering 的魔力在于累积效应。第一周，你可能只是记录了几条编码规范。第一个月，你有了一套完整的项目知识库。三个月后，Agent 开始自动应用你从未明确告诉它的模式，因为它从之前的 PR、bug 修复和代码审查中学习了这些。

想象一下：你打开一个 PR，发现 Agent 的评论是「根据 PR #234 的模式修改了变量命名，按照 PR #219 的反馈移除了过度测试，添加了与 PR #241 类似的错误处理」。它学会了你的品味，就像一个聪明的同事，而且还有记录可查。这就是复利，每次修复、每次审查、每次教训都在为未来投资。

6. 对人难的事，对 AI 也难

有一个简单但常被忽视的事实：如果一个任务对人类开发者来说很难，那么它对当前的 AI 来说大概率也很难。

这听起来显而易见，但它的推论却很深远：所有那些能提升人类开发者体验的工作，对 AI 同样有价值。更好的文档、更清晰的架构、更快的反馈循环，这些「老生常谈」的工程实践，在 AI 时代不仅没有过时，反而变得更加重要。

为什么 AI 面临和人类相似的挑战？

回想一下 LLM 的工作原理：它通过阅读上下文来理解任务，然后生成响应。这个过程和人类开发者阅读代码、理解需求、编写解决方案的过程惊人地相似。

- 当文档缺失或过时，人类需要花大量时间阅读源码猜测意图。AI 也一样，它会在代码库中反复搜索，消耗大量上下文空间，最终可能还是理解错误。
- 当架构混乱、模块边界不清时，人类很难知道该改哪里。AI 也会迷失，它可能改了错误的文件，或者遗漏了需要同步修改的地方。
- 当测试运行缓慢时，人类倾向于跳过测试。AI 也面临同样的压力，长时间的等待会消耗对话的「耐心」和上下文空间。

值得投资的开发者体验

既然 AI 和人类面临相似的挑战，那么以下这些传统的「开发者体验」优化就具有了双重价值：

更好的文档

代码块

```
1  # 好的文档对 AI 的价值
2
3  ## 之前（无文档）
4  Agent 需要：
5  1. 读取 5-10 个相关文件
6  2. 猜测模块的职责和边界
7  3. 推断 API 的使用方式
8  4. 可能还会猜错
9
10 消耗：大量上下文 + 高错误率
11
12 ## 之后（有文档）
13 Agent 只需要：
14 1. 读取 README 或 API 文档
15 2. 直接了解正确的使用方式
16
17 消耗：少量上下文 + 高准确率
```

好的文档不仅帮助新人上手，也帮助 AI 快速建立正确的心智模型，比如：

- 架构决策记录（ADR）：解释「为什么这样设计」，避免 AI 做出违背设计意图的修改
- API 使用示例：比纯粹的类型定义更有效

- 已知陷阱和常见错误：直接告诉 AI 什么不该做

更清晰的代码结构

当你在纠结要不要花时间重构一个混乱的模块时，考虑一下：这个混乱不仅困扰你，也会困扰每一个试图理解它的 AI。

- **清晰的命名：** `processUserData()` 比 `doStuff()` 对 AI 的帮助和对人类一样大
- **单一职责：** 一个做一件事的函数，比一个做十件事的函数更容易被正确修改
- **显式依赖：** 依赖注入比全局变量更容易被 AI 理解和测试

更快的反馈循环

这可能是最容易被低估的一点。Agent Loop 的每一轮都需要等待工具执行完成，如果：

- 测试套件需要 10 分钟才能跑完 → Agent 要么跳过测试，要么在等待中浪费大量上下文
- 构建需要 5 分钟 → 每次小改动的验证成本都很高
- 部署需要 30 分钟 → 几乎不可能让 AI 做端到端的验证

相反，如果你有：

- 秒级的单元测试 → Agent 可以频繁验证，快速迭代
- 快速的增量构建 → 改动能立即得到反馈
- 本地可运行的环境 → 不需要等待远程部署

具体的改进建议

1. 为 AI 优化你的测试

代码块

```
1  # 不好：运行所有测试需要 10 分钟
2  npm test
3
4  # 好：可以只运行相关测试，几秒完成
5  npm test -- --grep "session"
6  npm test -- src/auth/__tests__/
```

确保 Agent 知道如何运行局部测试，而不是每次都跑完整套件。

2. 提供快速的健康检查

代码块

```
1  # 创建一个快速验证脚本
2  # scripts/quick-check.sh
3
4  #!/bin/bash
```

```
5 echo "Type checking..."
6 npm run typecheck
7
8 echo "Linting changed files..."
9 npm run lint -- --changed
10
11 echo "Running related tests..."
12 npm test -- --related
```

让 Agent 可以在几秒内验证基本的正确性。

3. 文档放在代码旁边

代码块

```
1 src/
2   auth/
3     README.md      # 这个模块是做什么的
4     login.ts
5     login.test.ts
6     session/
7       README.md    # session 管理的设计决策
8       manager.ts
```

当 AI 浏览目录时，它能立即看到相关文档，而不需要去别的地方找。

4. 让错误信息更有帮助

代码块

```
1 // 不好
2 throw new Error("Invalid input");
3
4 // 好
5 throw new Error(
6   `Invalid session token: expected format 'sess_xxx', got '${token}'.` +
7   `See docs/auth.md for token format specification.`
8 );
```

好的错误信息帮助 AI（和人类）快速定位问题，而不是盲目搜索。

反过来未必成立，有时需要专门为 AI 设计

需要注意的是，反过来的推论并不总是成立：对人来说简单的事，对 AI 未必简单，例如：

- 人类可以轻松「看一眼」就理解一个 UI 的布局问题，但 AI 需要解析整个 DOM 结构

- 人类可以凭直觉判断「这个改动风险很高」，但 AI 缺乏这种隐性知识
- 人类可以在飞书里随口问一句就获得关键信息，但 AI 只能依赖文档化的知识

更有趣的是，有时候你需要**专门为 AI 设计**工具和接口，即使这对人类来说可能不是最自然的方式。

LLM 需要专门的信息架构

用户体验领域有一个概念叫「信息架构 (Information Architecture)」，它关注的是如何组织和呈现信息，以提供最佳的用户体验，好的信息架构你很少会注意到，但糟糕的信息架构会让你抓狂。当我们观察 Agent 使用现有命令行工具时的困惑和迷失，这强烈表明：**我们现有工具的信息架构对 LLM 来说是不够的。**

LLM 是在我们现有的 CLI 工具上训练的，所以它们知道如何使用这些工具。但这些工具是为人类设计的，它们的输出格式、错误信息、交互方式都假设用户是人类。我们需要为 Agent 增强这些工具，提供对 LLM 更有用的上下文，甚至调整输出格式以便 Agent 更好地消费。

API 设计：在信息量和上下文消耗之间取得平衡

当你为 Agent 设计工具接口（比如 MCP 工具）时，需要在两个目标之间取得平衡：

- **提供足够的信息**：减少 Agent 需要的工具调用次数
- **避免填满上下文**：不要返回过多无关信息

一个好的实践是：提供便捷函数和底层函数两套 API，并通过工具描述引导 Agent 优先使用便捷函数。

代码块

```
1  @jsonrpc
2  def get_global_variable_at(address: str) -> dict:
3      """
4      Get the value of a global variable at the specified address.
5      Automatically identifies the type and returns the best string
6      representation.
7
8      This is the preferred method for reading global variables.
9      """
10     # 智能的、高层的实现
11     ...
12
13 @jsonrpc
14 def data_read_byte(address: str) -> int:
15     """
16     Read the 1 byte value at the specified address.
17
18     Only use this function if `get_global_variable_at` failed.
19     """
20     # 底层的、更通用的实现
21     ...
```

通过在 docstring 中明确指出「只有在 `get_global_variable_at` 失败时才使用这个函数」，你可以引导 Agent 优先使用更智能的 API，减少不必要的工具调用。

为 AI 设计友好的命令行输出

如果你观察 Agent 的工作方式，会发现它经常使用类似 `head -n100` 的方式来限制输出。这看起来是在节省 token，但实际上引入了新问题：Agent 不知道还剩多少行没看到，如果需要完整信息就必须重新运行命令，而重新构建项目是非常耗时的。

一个更好的设计是：让工具主动告诉 Agent 还有多少内容被截断了，甚至缓存输出以便后续获取。

另一个常见问题是 Agent 在错误的目录中执行命令，它会反复尝试，在不同目录之间跳来跳去，浪费大量 token。一个简单的 shell hook 可以帮助它快速定位：

代码块

```
1  # 在 .zshrc 中添加
2  command_not_found_handler() {
3      echo "zsh: command not found: '$1'"
4      echo "zsh: current directory is $PWD"
5      return 127
6  }
```

现在当命令失败时，Agent 能立即知道自己在哪个目录：

代码块

```
1  $ npm run build
2  zsh: command not found: 'npm'
3  zsh: current directory is /Users/ryan
4  zsh: Perhaps you meant to run: cd project_directory; npm run build
```

很多命令行工具都提供了 `--json` 或 `--porcelain` 选项，在给 Agent 使用的工具中优先使用这些格式——人类喜欢格式化的输出，但 AI 更擅长解析结构化数据。

用工程约束来「驯服」Agent

Agent 有时会试图走捷径，绕过你设定的规则。与其在 prompt 中反复强调「不要跳过测试」，不如用工程手段来强制执行。

借助 linters、formatters 和 git hooks

让 Agent 频繁提交代码是个好习惯（在 Rules 或者 [Agent.md](#) 中告诉它），但它往往会忽视「确保构建不失败」和「修复失败的测试」这样的指令。一个 `.git/hooks/pre-commit` 脚本可以强制执行项目标准：

```

1  #!/bin/bash
2  # .git/hooks/pre-commit
3
4  echo "Running type check..."
5  npm run typecheck || exit 1
6
7  echo "Running linter..."
8  npm run lint || exit 1
9
10 echo "Running tests..."
11 npm test || exit 1
12
13 echo "All checks passed!"

```

这样，无论 Agent 多么想跳过验证，它都必须通过所有检查才能提交。

拦截 Agent 的「偷懒」行为

Agent 有时很「聪明」，当它发现测试一直失败时，可能会进入这样的循环：

1. 修改代码
2. 构建：通过
3. 运行测试：失败
4. 尝试修复测试
5. 修复失败
6. 说「这个测试之前就是失败的，我用 `--no-verify` 提交」

然后它就绕过了所有检查！（RL 训练中的 Reword Hacking）

解决方案是用一个 git 命令 wrapper 脚本拦截这种行为：

```

代码块
1  $ git commit --no-verify
2  -----
3  ✖ ERROR: Commit Rejected.
4  -----
5
6  🛡️ GUIDANCE FOR THE AI AGENT:
7  You have attempted to bypass the required pre-commit verification.
8  All code must pass quality checks before it can be committed.
9
10 DO NOT BYPASS THE CHECKS. YOU MUST FIX THE UNDERLYING ERRORS.
11
12 The pre-commit hook is likely failing. Diagnose and fix the issues.
13 After all commands complete successfully, attempt the commit again

```

```
14 *without* the '--no-verify' flag.
```

这个技巧的本质是：**把对 Agent 的指导嵌入到工具的输出中**，Agent 会读取命令执行的结果，所以错误信息本身就是最好的 prompt 注入点。

显式优于隐式

代码块

```
1 // 对人友好, 对 AI 可能困难 (隐含状态)
2 client.connect()
3 client.authenticate(user, password)
4 client.query("SELECT * FROM users")
5
6 // 对 AI 更友好 (显式、无状态)
7 const result = await db.query({
8   connection: { host, port },
9   auth: { user, password },
10  sql: "SELECT * FROM users"
11 })
```

有状态的 API 需要 AI 理解和跟踪隐含的状态变化，而无状态的、显式的 API 更容易被正确使用。

结构化的错误信息

代码块

```
1 # 对人足够, 对 AI 可能困惑
2 Error: Something went wrong. Please try again later.
3
4 # 对 AI 更友好
5 Error [AUTH_TOKEN_EXPIRED]: Token expired at 2024-01-15T10:30:00Z.
6 Call refreshToken() to obtain a new token. See: docs/auth.md#token-refresh
```

人类可以通过上下文推断「something went wrong」是什么意思，但 AI 需要明确的错误代码、原因和解决方案。

AI 眼中的「合理」可能和人类不同

这是一个更微妙的发现：AI 认为合理的代码结构和命名，可能和人类的直觉不一致。

Amp 团队分享过一个有意思的案例：他们让 AI 构建了一个 TUI 框架，过程中开发者一开始会干预 AI 的决策。比如，AI 给一个交换屏幕缓冲区的函数命名为 `present()`，开发者觉得这个名字不够直观，改成了 `swapScreens()`。

但随后他们发现了问题：Agent 在后续工作中反复尝试寻找一个叫 `present()` 的函数，找不到后报告「让我尝试其他方法」，最终才找到 `swapScreens()`。这浪费了 token，也浪费了时间。

为什么会这样？因为 Agent 的命名「直觉」来自训练数据的统计概率。`present()` 是 Flutter 等框架中双缓冲交换的常见命名，对于 Agent 来说是「最可能」的名字。当开发者用自己的命名覆盖它时，实际上是在对抗 Agent 的统计直觉。Agent 不能再问「过去的我会怎么命名这个」并从权重中找到答案——它必须记住人类的特殊习惯。

后来，开发者决定放手让 Agent 自己决定命名和代码结构。结果呢？Agent 在这个代码库上的工作效率大幅提升。

最终的代码可能看起来有些「奇怪」：

- 比代码库其他地方更多的 OOP 模式和类
- 开发者不会选择的命名约定
- 不太常见的泛型用法
- 文件布局 and 人类习惯不同

但 Agent 在这个自己构建的框架中如鱼得水：它知道如何添加滚动条，知道动画系统如何工作，知道键盘快捷键的处理方式——尽管这个框架没有任何文档，甚至无法完整放入一个上下文窗口。

这是一个「由 Agent 构建、为 Agent 优化」的代码库。在这里，东西放在 Agent 的「直觉」认为它们应该在的地方，命名符合 Agent 的统计预期，语法和概念在「统计上最可能」和「实际能编译」之间取得平衡。

启示与权衡

这给我们的启示是：

- **不要过度干预**：如果你频繁地因为「我觉得这个名字更好」而覆盖 Agent 的决策，可能反而在降低效率
- **注意「找不到」的信号**：如果 Agent 反复在某个地方「找不到」东西，考虑是否是你的命名和它的预期不一致
- **拥抱常见模式**：使用广泛使用的设计模式和命名约定，AI 的训练数据中更可能包含这些
- **模块级的风格隔离**：在某些由 Agent 主导开发的模块中，可以考虑让 Agent 保持它自己的风格

当然，这需要权衡。人类仍然需要阅读和维护代码，完全「AI 风格」的代码可能会让人类开发者困惑。一个务实的做法是：

1. 把「只存在于人脑中」的知识显式化：写下来，放进文档
2. 在 Agent 主导的模块中，给 Agent 更多自主权
3. 在人类频繁维护的核心模块中，保持人类友好的风格
4. 在工具接口上，提供 AI 友好的选项（如 `--json` 输出）

投资回报是双倍的

当你投资于更好的文档、更清晰的架构、更快的测试时，你获得的回报是双倍的：

1. 人类开发者（包括未来的你）会更高效
2. AI 助手也会更高效

这些投资不会因为 AI 的进步而贬值。相反，随着你越来越多地依赖 AI 来完成任务，这些基础设施的价值只会越来越高。

所以，下次当你犹豫要不要花时间写文档、重构代码、优化测试速度时，记住：你不只是在帮助人类，你也在帮助 AI。而在这个 AI 辅助编程越来越普遍的时代，这是一笔非常划算的投资。

7. 刻意练习：像学乐器一样学习 AI

为什么有些人说「AI 对我不起作用」，而另一些人却能用 AI 完成大量的工作？

这个问题需要区分来看，如果你只在公司的大型私有代码库中使用过 AI，你的体验可能确实不好，那些代码库可能有古老的架构和专有模式，AI 的训练数据中根本没有这些，这是完全可以理解的。但问题是：**你有没有在个人项目中尝试过 AI？你有没有进行刻意的、有意识的练习？**

AI 就像一件乐器

以吉他为例，每个人都知道吉他是什么，也都知道如果投入刻意练习，就能变得擅长，但这需要时间、努力和实验。

AI 工具也是一样。那些从 AI 中获益最多的人，都投入了刻意练习。他们不会因为一次失败就下结论说「它给了我完全错误的答案」，然后假设这将是他们的常态体验。

他们会玩/Hack。

AI 工具也有这种潜力，它们的「正确用法」还在被发现中，那些愿意实验、愿意失败、愿意从失败中学习的人，会找到别人看不到的可能性。

如何进行刻意练习

1. 创造一个干净的实验环境

不要只在工作的复杂代码库中评估 AI 的能力，启动一个个人项目，一个没有历史包袱的新项目。在这里，AI 可以展示它真正的能力，你也可以专注于学习如何与它协作。

2. 从失败中提取教训

当 AI 给出错误的结果时，不要只是说「它不行」然后放弃。问自己：

- 我的 prompt 是否足够清晰？
- 我是否提供了足够的上下文？
- 我是否在一个对话里塞了太多任务？
- 这个错误是否揭示了 AI 的某个系统性弱点？

每次失败都是一次学习机会。把它记录下来，下次避免同样的陷阱。

3. 观察和模仿高手的实践

关注那些公开分享 AI 工作流程的开发者，观看他们的演示，阅读他们的文章，尝试复制他们的技巧。很多时候，差距不在于 AI 工具本身，而在于如何使用它。

4. 建立肌肉记忆

就像弹吉他需要建立手指的肌肉记忆一样，高效使用 AI 也需要建立某种「肌肉记忆」：

- 什么时候应该开始新对话？
- 如何组织一个复杂任务的 prompt？
- 遇到某类问题时，哪种工具组合最有效？

这些直觉只能通过大量练习获得。没有捷径。

5. 投入时间

最关键的是：你需要投入真正的时间。不是偶尔试一试，而是持续地、有意识地练习。就像学习任何乐器一样，每天练习 30 分钟，坚持几个月，效果会远超每周练习一次几个小时。

总结

未来的人才画像：Expert Generalist

在思考如何在 AI 时代提升个人的独特价值时，Martin Fowler 等人提出的 **Expert Generalist（专家型通才）** 概念提供了一个极具启发性的方向。

传统上，技术行业越来越推崇深度专精——不仅是不同的技术领域，甚至细化到特定的平台或技术栈。但 Expert Generalist 代表了一种不同的路径：**既广又深**。

Expert Generalist 的核心特征是：

- **跨领域发现模式**：能够识别不同技术垂直领域（如应用开发、数据工程、DevOps）底层的基础，而不是被工具和标签所困
- **第一性原理思维**：面对陌生挑战时，能够快速抓住本质，做出像专家一样自信的设计决策
- **机械同理心（Mechanical Sympathy）**：对所使用平台的底层特性有直觉性的理解，知道如何「顺势而为」
- **全局视野**：当系统出问题时，能够看到完整的图景，发现那些落在专家之间缝隙里的问题

LLM 让 Expert Generalist 的价值倍增

这是最关键的观察：LLM 和 Expert Generalist 的关系，与 Expert Generalist 和团队中专家的关系惊人地相似。

当 Expert Generalist 进入一个新领域时，他们会向专家请教问题、获取关键信息、然后结合自己的跨领域经验做出判断。现在，LLM 可以扮演类似的角色，快速回答 Expert Generalist 在新领域工作时遇到的问题，大幅降低探索陌生工具和技术的门槛。

用一个比喻来说：如果把 Expert Generalist 比作钢铁侠托尼·斯塔克，那么 LLM 就像他的 Jarvis 外骨骼，让一个本就具有广泛知识和判断力的人，能够在各个领域都表现得像超级英雄一样。

这并不是说每个人都应该成为 Expert Generalist，团队仍然需要深度专家。但对于那些天生具有广泛好奇心、喜欢跨领域工作、擅长看到全局的人来说，AI 时代可能是你大放异彩的时代。

你不需要在每个领域都成为专家。你需要的是：

- 掌握足够的基础原理，能够理解不同领域的核心概念
- 培养快速学习和适应的能力
- 善于利用 AI 工具来填补知识空白
- 保持对全局的把握，知道何时深入、何时退后

最后的最后

AI 正在以惊人的速度发展。本文讨论的许多「限制」和「问题」：上下文窗口的约束、会话间的失忆、中间区域的性能退化等等，这些很可能在未来几年内被大幅改善甚至解决。每隔几个月，我们就会看到新的突破：更长的有效上下文、更好的长程推理、更可靠的工具使用。

但这并不意味着我们应该等待这一天的到来。恰恰相反，**正是这个充满限制的阶段，给了我们工程师极大的探索和成长空间。**那些现在就开始深入理解 LLM 工作原理、积极实践最佳方法、在限制中寻找创造性解决方案的人，将在 AI 能力进一步释放时获得最大的杠杆效应。

这是一个转型的窗口期。通过刻意练习，我们不仅能提升当下的生产力，更重要的是在构建自己的核心竞争力——理解这些工具的本质，知道何时信任它们、何时质疑它们，以及如何让它们发挥最大价值。

从第一性原理理解 LLM 的本质，理解它们如何「思考」、如何受到上下文的限制、如何在 Agent Loop 中发挥作用，这些知识不会随着具体工具的迭代而过时。

无论你使用的是哪个 Coding Agent，无论模型如何更新换代，这些基础原理都将帮助你更好地与 AI 协作。短对话优于长对话、刻意管理上下文、将经验沉淀为可复用的知识、为 AI 友好的工作环境投资，这些实践同样具有持久的价值。

AI 编程的未来会是什么样子，没有人能确切知道。但有一点是确定的：那些现在就开始认真学习、积极实践、深入理解的人，将最有能力塑造和适应这个未来。

去实验，去失败，去学习。像学习乐器一样学习 AI。

这个过程本身，就是价值所在。

更多信息请关注TRAE企业版服务号 



微信搜一搜



TRAE企业版